Kotlin & Jetpack Compose Concepts Guide - Vol. 2

AppModule / DI deep-dive | Multiple Implementations
State Survival Levels | Flutter Comparisons

Picks up where Volume 1 left off.

All concepts shown in the context of the Online Store project.

Every section includes a Flutter/Dart side-by-side comparison.

Table of Contents

1. The ui/theme/ Folder
2. AppModule.kt - How Hilt Picks It Up Without Being Called
3. Multiple Implementations of the Same Interface
4. The @Named Qualifier
5. FakeProductRepositoryImpl - Showcase File Explained
6. State Survival in Android - The Five Levels
7. Level 1: remember { } - Composable-Local State
8. Level 2: ViewModel + StateFlow - Screen State
9. Level 3: @Singleton Repository - App-Wide State
10. Level 4: SavedStateHandle - Survive Process Death
11. Level 5: Room / DataStore - Truly Persistent
12. How the Cart Works in This App
13. Flutter vs Android State Management Comparison
14. Decision Guide: Which Level Should I Use?

1. The ui/theme/ Folder

When Android Studio creates a new Jetpack Compose project it generates a ui/theme/ folder with three files. These are not something you write - they are scaffolded for you. They define the global Material3 theme that wraps the entire app.

The three files

Color.kt

Defines the app's color palette. Contains two color schemes: one for light mode and one for dark mode. Every time you write MaterialTheme.colorScheme.primary in a composable, the color resolves from here.

```
// Color.kt (auto-generated, you can customize the values)
val Purple80 = Color(0xFFD0BCFF)
val PurpleGrey80 = Color(0xFFCCC2DC)

private val DarkColorScheme = darkColorScheme(
    primary = Purple80,
    secondary = PurpleGrey80,
)
private val LightColorScheme = lightColorScheme(
    primary = Purple40,
    secondary = PurpleGrey40,
)
```

Type.kt

Defines the typography scale: display, headline, title, body, label sizes and weights. Every time you write MaterialTheme.typography.titleMedium in a composable, the font style resolves from here.

```
// Type.kt (auto-generated)
val Typography = Typography(
    bodyLarge = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp,
    ),
    // ... more text styles
)
```

Theme.kt

Combines Color.kt and Type.kt into a single MaterialTheme wrapper composable. This is the only file from this folder that you directly reference - it wraps your entire app in MainActivity.

```
// Theme.kt (auto-generated)
@Composable
fun OnlinestoreclaudeTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit,
) {
    val colorScheme = if (darkTheme) DarkColorScheme else LightColorScheme
```

```
    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content,
    )
}

// MainActivity.kt -- the one place it is used
setContent {
    OnlinestoreclaudeTheme {   // <-- wraps everything
        AppNavigation()
    }
}
```

[NOTE] Flutter equivalent: ThemeData inside MaterialApp(theme: ThemeData(...)). The ui/theme/ folder is the Compose version of that ThemeData configuration.

2. AppModule.kt - How Hilt Picks It Up Without Being Called

AppModule is never imported or referenced by name anywhere in the app's runtime code. Yet it is essential. This feels confusing at first, so here is exactly what happens.

What happens at compile time

KSP (Kotlin Symbol Processing) runs during every Gradle build. It scans every class in the project for Hilt annotations (@Module, @HiltViewModel, @Inject, etc.) and generates Java/Kotlin glue code in the build/ folder. This generated code is what actually wires everything together at runtime.

```
// What you write
@Module
@InstallIn(SingletonComponent::class)
abstract class AppModule {
    @Binds @Singleton
    abstract fun bindProductRepository(impl: ProductRepositoryImpl): IProductRepository
}

// What KSP generates for you (simplified, in build/generated/)
// You never see or touch this, but it is what runs:
public final class AppModule_BindProductRepositoryFactory {
    public static IProductRepository bindProductRepository(ProductRepositoryImpl impl) {
        return impl;
    }
}
```

The chain from annotation to injection

```
Step 1: You annotate AppModule with @Module + @InstallIn(SingletonComponent::class)
        -> KSP registers these bindings into the SingletonComponent

Step 2: You annotate ProductRepositoryImpl with @Inject constructor()
        -> KSP generates a Factory that knows how to create ProductRepositoryImpl

Step 3: You annotate ProductsViewModel with @HiltViewModel + @Inject constructor(repo: IProductRepository)
        -> KSP sees "needs IProductRepository" -> looks up SingletonComponent -> finds AppModule binding
        -> generates a ViewModel factory that calls ProductRepositoryImpl factory first

Step 4: You call hiltViewModel() in ProductsScreen
        -> at runtime, Hilt uses the generated factory to build ProductsViewModel
        -> ProductRepositoryImpl is created (or reused if already exists as @Singleton)
        -> passed into ProductsViewModel constructor automatically
```

[NOTE] This is annotation-based DI - you declare WHAT you need, Hilt figures out HOW to build it. Flutter's DependenciesScope requires you to manually wire everything. Hilt automates that wiring.

Why it is abstract and uses @Binds instead of @Provides

@Binds is used when the implementation already has @Inject constructor - Hilt already knows how to build it, so you just need to tell it which interface it satisfies. @Binds generates less code than @Provides and is preferred when possible.

```
// @Binds - use when the impl has @Inject constructor (preferred, less generated code)
@Binds @Singleton
abstract fun bindCartRepository(impl: CartRepositoryImpl): ICartRepository

// @Provides - use when you need to manually construct the object
// (e.g. third-party classes you cannot annotate with @Inject)
@Provides @Singleton
fun provideRetrofit(): Retrofit {
    return Retrofit.Builder()
        .baseUrl("https://api.example.com/")
        .addConverterFactory(GsonConverterFactory.create())
        .build()
}

// You cannot use @Binds for Retrofit because Retrofit is a third-party class
// and you cannot add @Inject to its constructor.
```

3. Multiple Implementations of the Same Interface

A core principle of clean architecture is that you code to interfaces, not implementations. This makes it easy to swap implementations without touching any of the code that uses them. A common real-world scenario:

```
interface IProductRepository {
    suspend fun getProducts(): List<Product>
    suspend fun getProductById(id: Int): Product?
    suspend fun searchProducts(query: String, category: Category): List<Product>
}

// Implementation 1: talks to a real REST API via Retrofit
class ProductRepositoryImpl @Inject constructor(
    private val apiService: ApiService,
) : IProductRepository { ... }

// Implementation 2: returns hardcoded data -- no network needed
class FakeProductRepositoryImpl @Inject constructor() : IProductRepository { ... }
```

The problem: Hilt does not know which one to use

If you register both in AppModule without any qualifier, Hilt throws a compile-time error:

```
// This FAILS to compile:
@Binds abstract fun bindReal(impl: ProductRepositoryImpl): IProductRepository
@Binds abstract fun bindFake(impl: FakeProductRepositoryImpl): IProductRepository

// Error: [Dagger/DuplicateBindings] IProductRepository is bound multiple times.
```

[!] This error happens at COMPILE time, not at runtime. Hilt catches it before the app even runs - this is one of its biggest advantages over manual DI.

The solution: @Named qualifier

Tag each binding with a string label so Hilt can tell them apart:

```
@Module
@InstallIn(SingletonComponent::class)
abstract class AppModule {

    @Binds @Singleton @Named("real")
    abstract fun bindProductRepository(impl: ProductRepositoryImpl): IProductRepository

    @Binds @Singleton @Named("fake")
    abstract fun bindFakeProductRepository(impl: FakeProductRepositoryImpl): IProductRepository
}
```

4. The @Named Qualifier

@Named is used on both the binding (in AppModule) and the injection point (constructor parameter in ViewModel). The string must match exactly.

Using @Named in ViewModels

```
// ProductsViewModel.kt
@HiltViewModel
class ProductsViewModel @Inject constructor(
    @Named("real") private val productRepository: IProductRepository,
    // Hilt sees @Named("real") -> looks for a binding tagged "real" -> finds ProductRepositoryImpl
) : ViewModel()

// To switch to fake data during development, change one word:
@HiltViewModel
class ProductsViewModel @Inject constructor(
    @Named("fake") private val productRepository: IProductRepository,
    // Now Hilt injects FakeProductRepositoryImpl instead
) : ViewModel()
```

[NOTE] This is the power of interface-based DI: swapping from real to fake data is a one-word change in the ViewModel. Zero changes to the screen composables, zero changes to the repository implementation itself.

Custom qualifier annotation (cleaner alternative to @Named)

@Named uses plain strings which can be mistyped. A custom qualifier annotation is type-safe and checked by the compiler:

```
// Define your qualifiers once
@Qualifier
@Retention(AnnotationRetention.BINARY)
annotation class RealDataSource

@Qualifier
@Retention(AnnotationRetention.BINARY)
annotation class FakeDataSource

// Use them in AppModule
@Binds @Singleton @RealDataSource
abstract fun bindReal(impl: ProductRepositoryImpl): IProductRepository

@Binds @Singleton @FakeDataSource
abstract fun bindFake(impl: FakeProductRepositoryImpl): IProductRepository

// Use them in ViewModel
@HiltViewModel
class ProductsViewModel @Inject constructor(
    @RealDataSource private val productRepository: IProductRepository,
) : ViewModel()
```

```
// Typo with @Named("rael") compiles fine but crashes at runtime.
// Typo with a custom annotation fails at compile time - safer.
```

ICartRepository and IOrderRepository have no @Named -- why?

Only IProductRepository has two implementations in this project. ICartRepository and IOrderRepository each have exactly one implementation, so Hilt knows which class to inject without any qualifier. @Named is only needed when the same interface has more than one binding registered.

```
// No @Named needed -- only one binding exists for each
@Binds @Singleton
abstract fun bindCartRepository(impl: CartRepositoryImpl): ICartRepository

@Binds @Singleton
abstract fun bindOrderRepository(impl: OrderRepositoryImpl): IOrderRepository
```

5. FakeProductRepositoryImpl - Showcase File Explained

FakeProductRepositoryImpl is a reference/showcase file. It is wired into AppModule with @Named("fake") but nothing currently injects @Named("fake"). Its purpose is to demonstrate the multiple-implementation pattern and serve as a ready-made starting point for testing.

Three practical uses for a fake repository

1. UI development without a backend

When the backend API is not ready yet, switch the ViewModel to @Named("fake") and build the entire UI against hardcoded data. No network, no authentication, no errors.

2. Unit testing

In unit tests you do not use Hilt at all. You simply pass the fake implementation directly into the ViewModel constructor:

```
// ProductsViewModelTest.kt
class ProductsViewModelTest {

    @Test
    fun `load() emits Completed state with products`() = runTest {
        // Pass the fake directly - no Hilt, no AppModule, no @Named needed in tests
        val fakeRepo = FakeProductRepositoryImpl()
        val viewModel = ProductsViewModel(productRepository = fakeRepo)

        viewModel.load()

        val state = viewModel.state.value
        assertTrue(state is ProductsState.Completed)
    }
}
```

3. Screenshot / UI tests

In instrumented UI tests (Espresso / Compose Test), you can replace the real binding with the fake one using a Hilt test module so screens always render with predictable data.

```
// In your androidTest folder
@UninstallModules(AppModule::class)   // remove the real AppModule
@HiltAndroidTest
class ProductsScreenTest {

    @Module @InstallIn(SingletonComponent::class)
    abstract class FakeAppModule {
        @Binds @Singleton @Named("real")   // same qualifier name, different impl
        abstract fun bindProductRepository(impl: FakeProductRepositoryImpl): IProductRepository
    }
}
```

6. State Survival in Android - The Five Levels

This is one of the most important concepts to understand in Android development. Unlike Flutter where state either lives in a Widget or in a Provider/InheritedWidget, Android has five distinct levels of state persistence, each with a different lifetime.

The question to ask is always: how long does this data need to stay alive?

Overview table

Level	Survives rotation?	Survives nav back?	Survives process death	Survives restart?
remember { }	No	No	No	No
ViewModel	Yes	No	No	No
@Singleton repo	Yes	Yes	No	No
SavedStateHandle	Yes	Yes	Yes	No
Room / DataStore	Yes	Yes	Yes	Yes

[NOTE] In Flutter, state either lives in a Widget (lost on pop) or in a root-level Provider (lives for the app session). Android has the same two extremes but adds three intermediate levels that Flutter does not have out of the box.

7. Level 1: remember { } - Composable-Local State

remember stores a value in the composition. It survives recompositions (redraws) but is destroyed the moment the composable leaves the composition tree - which happens when the user navigates away.

```
@Composable
fun ProductsScreen(viewModel: ProductsViewModel = hiltViewModel()) {

    // These exist only while ProductsScreen is visible
    var searchQuery by remember { mutableStateOf("") }
    var selectedCategory by remember { mutableStateOf(Category.ALL) }

    // When user navigates to CartScreen and comes back:
    // searchQuery resets to ""
    // selectedCategory resets to Category.ALL
}
```

rememberSaveable - upgrades remember to survive rotation AND process death

rememberSaveable works like remember but saves its value into a Bundle (the same mechanism Android uses for onSaveInstanceState). Use it for lightweight UI state like scroll position, selected tab, or text field input that should not reset on screen rotation.

```
@Composable
fun ProductsScreen() {

    // Survives screen rotation -- resets when user navigates away and comes back
    var searchQuery by rememberSaveable { mutableStateOf("") }

    // For custom data classes, implement Parcelize or provide a custom Saver:
    // var filter by rememberSaveable(stateSaver = FilterSaver) { mutableStateOf(Filter()) }
}
```

[NOTE] Flutter equivalent of remember: a local variable in a State class. Flutter equivalent of rememberSaveable: storing state in a StatefulWidget and relying on the OS to preserve it - which Flutter does not do automatically for nav pops.

8. Level 2: ViewModel + StateFlow - Screen State

ViewModel survives screen rotation (configuration changes) because it is stored in the ViewModelStore, which is separate from the Activity/Fragment lifecycle. However it is destroyed when the screen is permanently removed from the back stack (user pressed Back, or popBackStack() was called).

```
@HiltViewModel
class CartViewModel @Inject constructor(
    private val cartRepository: ICartRepository,
) : ViewModel() {

    private val _state = MutableStateFlow<CartState>(CartState.Initial)
    val state: StateFlow<CartState> = _state.asStateFlow()

    fun load() { ... }

    // onCleared() is called when the ViewModel is permanently destroyed
    // (screen removed from back stack). Use it to cancel subscriptions, close streams, etc.
    override fun onCleared() {
        super.onCleared()
        // cleanup if needed
    }
}
```

ViewModel lifetime with navigation back stack

```
User opens CartScreen
    -> NavBackStackEntry created for "cart" route
    -> CartViewModel created and attached to that NavBackStackEntry

User rotates device
    -> Activity recreated, but NavBackStackEntry survives
    -> Same CartViewModel instance reused (not recreated)
    -> State is preserved automatically

User presses Back (popBackStack)
    -> NavBackStackEntry for "cart" is removed
    -> CartViewModel.onCleared() is called
    -> CartViewModel is garbage collected
    -> All StateFlow values are gone

User navigates to CartScreen again
    -> New NavBackStackEntry created
    -> Brand new CartViewModel created
    -> LaunchedEffect(Unit) fires -> load() is called -> data loaded from repository
```

[NOTE] This is why load() is called inside LaunchedEffect(Unit) in CartScreen. Every time the user arrives at the screen, a fresh ViewModel is created and needs to load data from the repository. The repository is the durable store; the ViewModel is just a temporary window into it.

9. Level 3: @Singleton Repository - App-Wide State

A @Singleton class is created once and reused for the entire app lifetime. It lives from the first time it is requested until the app process is killed by Android. This is the closest equivalent to Flutter's root-level Provider or a global inherited widget.

```
@Singleton   // <-- one instance, never destroyed while the app is running
class CartRepositoryImpl @Inject constructor() : ICartRepository {

    // This list persists across all navigation events because the
    // CartRepositoryImpl object itself is never recreated.
    private val _cartItems = mutableListOf<CartItem>()

    override fun addToCart(product: Product) {
        val existing = _cartItems.find { it.product.id == product.id }
        if (existing != null) {
            val index = _cartItems.indexOf(existing)
            _cartItems[index] = existing.copy(quantity = existing.quantity + 1)
        } else {
            _cartItems.add(CartItem(product = product, quantity = 1))
        }
    }

    override fun getCartItems(): List<CartItem> = _cartItems.toList()
}
```

Why @Singleton on both the class AND in AppModule?

```
// CartRepositoryImpl.kt
@Singleton   // (1) scope annotation on the class itself
class CartRepositoryImpl @Inject constructor() : ICartRepository

// AppModule.kt
@Binds
@Singleton   // (2) scope annotation on the binding
abstract fun bindCartRepository(impl: CartRepositoryImpl): ICartRepository

// Having both is redundant but harmless.
// In practice, only (2) in AppModule is required when using @Binds.
// (1) on the class is ignored by Hilt when a @Binds binding also declares @Singleton.
// Keeping (1) is useful as documentation - a reader sees the class is intended to be singleton.
```

Flutter comparison

```
// Flutter - manual global state via root Provider
void main() {
  runApp(
    Provider<CartRepository>(
      create: (_) => CartRepositoryImpl(),  // one instance at root
```

```
      child: MyApp(),
    ),
  );
}

// Access anywhere:
final cart = context.read<CartRepository>();

// Android - same concept, done automatically by Hilt
// @Singleton in Hilt = Provider at the root of the widget tree
// hiltViewModel() = context.read<T>()
// No manual wiring, no BuildContext needed
```

10. Level 4: SavedStateHandle - Survive Process Death

The Android OS can kill your app's process at any time when it is in the background (to free up memory for other apps). When the user returns to your app, Android restores the UI but your in-memory objects are gone. SavedStateHandle survives this by serializing values into a Bundle before the process dies.

What process death looks like

```
Normal flow (no process death):
    User opens app -> CartRepositoryImpl created (@Singleton) -> User adds products
    User switches to another app (briefly) -> User returns -> CartRepositoryImpl still in memory

Process death flow:
    User opens app -> CartRepositoryImpl created -> User adds products
    User switches to another app for a long time
    Android kills the process to free memory  <-- CartRepositoryImpl is GONE
    User returns -> Android restores the UI (back stack preserved)
    BUT CartRepositoryImpl is recreated empty -> Cart is empty!

    This is why for truly important data (real cart, user session, etc.)
    you need Room or DataStore (Level 5).
```

Using SavedStateHandle in a ViewModel

Hilt automatically injects SavedStateHandle into any ViewModel that declares it. Use it for lightweight screen state like selected index, scroll position, form values - NOT for large lists of data.

```
@HiltViewModel
class ProductsViewModel @Inject constructor(
    @Named("real") private val productRepository: IProductRepository,
    private val savedStateHandle: SavedStateHandle,  // Hilt injects this automatically
) : ViewModel() {

    // saveable delegate: persists through rotation AND process death
    var searchQuery by savedStateHandle.saveable { mutableStateOf("") }
    var selectedCategoryIndex by savedStateHandle.saveable { mutableStateOf(0) }

    // Regular StateFlow: lost on process death
    private val _state = MutableStateFlow<ProductsState>(ProductsState.Initial)
    val state: StateFlow<ProductsState> = _state.asStateFlow()
}
```

[NOTE] Flutter has no direct equivalent of SavedStateHandle. The closest Flutter has is manually saving state to SharedPreferences before the app is paused - which you must implement yourself. Android's SavedStateHandle does this automatically.

11. Level 5: Room / DataStore - Truly Persistent

Room and DataStore write data to disk. Data survives everything: process death, app restart, and even device restart. Use these when data must not be lost under any circumstance.

DataStore - for simple key-value data

DataStore is the modern replacement for SharedPreferences. Good for: user settings, auth tokens, onboarding flags, last-selected tab.

```
// Create the DataStore (once, at app level)
val Context.dataStore by preferencesDataStore("user_prefs")

// Write
suspend fun saveAuthToken(token: String) {
    val key = stringPreferencesKey("auth_token")
    context.dataStore.edit { prefs -> prefs[key] = token }
}

// Read (returns a Flow - reactive, updates automatically)
fun getAuthToken(): Flow<String?> {
    val key = stringPreferencesKey("auth_token")
    return context.dataStore.data.map { prefs -> prefs[key] }
}
```

[NOTE] Flutter equivalent: SharedPreferences package or Hive for key-value storage.

Room - for structured/relational data

Room is an SQLite wrapper with full Kotlin coroutine and Flow support. Good for: cart items, order history, product cache, user data.

```
// 1. Define the table (Entity)
@Entity(tableName = "cart_items")
data class CartItemEntity(
    @PrimaryKey val productId: Int,
    val productName: String,
    val price: Double,
    val quantity: Int,
)

// 2. Define the queries (DAO)
@Dao
interface CartDao {
    @Query("SELECT * FROM cart_items")
    fun getAll(): Flow<List<CartItemEntity>>   // Flow = reactive, auto-updates UI

    @Upsert
    suspend fun upsert(item: CartItemEntity)
```

```
    @Delete
    suspend fun delete(item: CartItemEntity)

    @Query("DELETE FROM cart_items")
    suspend fun clearAll()
}

// 3. Define the database
@Database(entities = [CartItemEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun cartDao(): CartDao
}

// 4. Provide it via Hilt (in AppModule, using @Provides not @Binds)
@Provides @Singleton
fun provideDatabase(@ApplicationContext context: Context): AppDatabase {
    return Room.databaseBuilder(context, AppDatabase::class.java, "app_db").build()
}

@Provides @Singleton
fun provideCartDao(db: AppDatabase): CartDao = db.cartDao()
```

[NOTE] Flutter equivalent: sqflite or Drift (the Dart ORM similar to Room).

12. How the Cart Works in This App

The cart in this project uses Level 3 (@Singleton repository). Here is the complete flow showing exactly why data is preserved when navigating away and returning.

Step-by-step: adding a product and navigating back

```
1. App starts
   -> Hilt creates CartRepositoryImpl (@Singleton) and stores it
   -> _cartItems list is empty

2. User opens ProductDetailScreen
   -> ProductDetailViewModel is created
   -> Hilt injects CartRepositoryImpl (the SAME instance from step 1)

3. User taps "Add to Cart"
   -> viewModel.addToCart(product) is called
   -> cartRepository.addToCart(product) is called
   -> CartRepositoryImpl._cartItems now has 1 item
   -> The item is stored in the Singleton object in memory

4. User presses Back -> ProductDetailViewModel is destroyed
   -> CartRepositoryImpl is NOT destroyed (it is @Singleton)
   -> _cartItems still has 1 item

5. User taps the cart icon -> CartScreen opens
   -> CartViewModel is created (brand new)
   -> Hilt injects CartRepositoryImpl (the SAME instance from step 1)
   -> LaunchedEffect(Unit) fires -> cartViewModel.load() is called
   -> cartRepository.getCartItems() returns the list with 1 item
   -> CartState.Completed(items = [product]) emitted
   -> UI shows the product in the cart
```

Why this would break without @Singleton

```
// If CartRepositoryImpl had NO @Singleton:
//   Hilt would create a new CartRepositoryImpl for EVERY ViewModel that asks for it.
//   ProductDetailViewModel would get instance A (adds product to A._cartItems)
//   CartViewModel would get instance B (B._cartItems is empty)
//   The cart would always appear empty.

// @Singleton ensures:
//   ProductDetailViewModel and CartViewModel get the EXACT SAME instance
//   Any mutation in one is immediately visible in the other
```

Limitation: does not survive process death

Since the cart lives in memory, Android killing the process empties the cart. For a production app, persist the cart to Room so it survives:

```
// Production CartRepositoryImpl would look like:
class CartRepositoryImpl @Inject constructor(
    private val cartDao: CartDao,   // Room DAO injected by Hilt
) : ICartRepository {

    // Returns a reactive Flow - UI updates automatically when data changes
    override fun getCartItems(): Flow<List<CartItem>> {
        return cartDao.getAll().map { entities -> entities.map { it.toDomain() } }
    }

    override suspend fun addToCart(product: Product) {
        val existing = cartDao.findById(product.id)
        val newQty = (existing?.quantity ?: 0) + 1
        cartDao.upsert(CartItemEntity(product.id, product.name, product.price, newQty))
    }
}
```

13. Flutter vs Android State Management Comparison

A complete side-by-side for every state scenario you encounter:

Scenario	Flutter	Android / Jetpack
Local UI state (dropdown open, tab selected)	setState(() {}) inside a StatefulWidget	remember { mutableStateOf() } inside a @Composable
Local state that survives rotation	No built-in solution, need manual work	rememberSaveable { mutableStateOf() }
Screen state (loading/error/data)	StatefulWidget + setState or a local Controller	ViewModel + MutableStateFlow<State>
Shared state across screens (same session)	Root Provider or InheritedWidget at app root	@Singleton class via Hilt
Access shared state from a Widget	context.read<T>() or DependenciesScope.of(context)	hiltViewModel() or @Named injection in ViewModel
State that survives background process kill	No built-in solution	SavedStateHandle in ViewModel
Persistent key-value data	SharedPreferences / Hive package	DataStore (Preferences)
Persistent structured/relational data	sqflite / Drift package	Room database
Reactive data (auto-update UI on change)	StreamBuilder / ChangeNotifier + ListenableBuilder	Flow / StateFlow + collectAsState()
One-time initialization when screen appears	initState()	LaunchedEffect(Unit) { }
Re-initialize when a key value changes	didUpdateWidget()	LaunchedEffect(key) { }
Cleanup when screen is destroyed	dispose()	ViewModel.onCleared() or DisposableEffect

14. Decision Guide: Which Level Should I Use?

Run through these questions top to bottom when deciding where to put state:

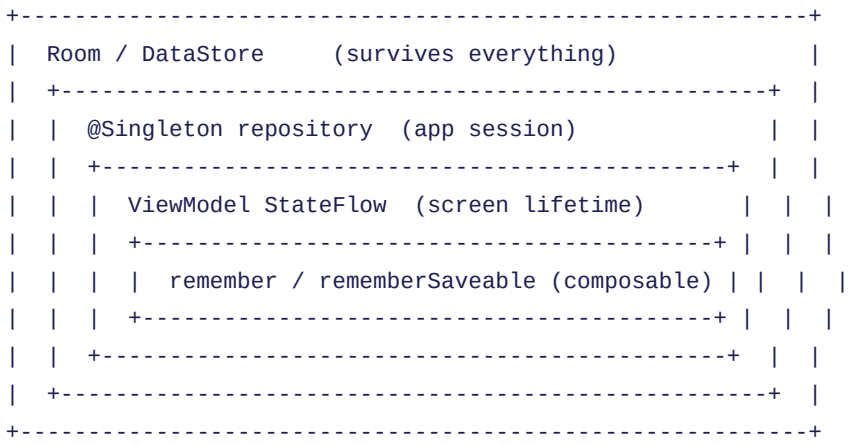
```
Q1: Does this data need to be shared between two or more screens?
    YES -> Use @Singleton repository (Level 3) or Room/DataStore (Level 5)
    NO  -> Continue to Q2

Q2: Does this data need to survive screen rotation?
    YES -> Use ViewModel StateFlow (Level 2) or rememberSaveable (Level 1b)
    NO  -> Use remember { } (Level 1) -- simplest option, use this by default

Q3: Does this data need to survive Android killing the process?
    YES and it is small (a string, an int, a simple flag)
        -> Use SavedStateHandle in ViewModel (Level 4)
    YES and it is a list or complex object
        -> Use Room or DataStore (Level 5)
    NO  -> ViewModel StateFlow (Level 2) is enough

Q4: Does this data need to survive app restarts?
    YES -> Room (structured data) or DataStore (key-value) -- Level 5
    NO  -> Any of the above levels is fine
```

Applied to this project

```
Cart items list
  -> Needs to be shared (ProductDetail adds, Cart reads) -> @Singleton repository
  -> In production: also persists to Room

Search query / selected category
  -> Local to ProductsScreen, no sharing needed
  -> Survives rotation? Nice to have but not critical
  -> Use rememberSaveable { mutableStateOf("") } in the composable

Loading / error / completed state
  -> Belongs to a single screen, needs to survive rotation
  -> Use ViewModel StateFlow

Order history
  -> Needs to persist across app restarts
  -> Use Room database

User auth token
  -> Key-value, needs to persist across restarts
  -> Use DataStore
```

Quick mental model

Think of state survival as concentric circles. Each circle is bigger (lives longer) than the one inside it:



Data flows inward (ViewModel reads from Repository reads from Room)
Events flow outward (UI calls ViewModel calls Repository writes to Room)